

Django Vue Lab

Занятие 3. JavaScript

Черепанов И.Н. Филиппев В.А.

Содержание

- JavaScript: назначение, типы данных, JSON
- Переменные: var, let, const, строгий режим
- Функции: объявление, стрелочные функции, this, замыкания
- Операторы: арифметика, строки, сравнения, логические выражения
- Работа с API: fetch, Promise, async/await
- DOM: дерево документа, поиск и изменение элементов
- События: обработчики, всплытие, делегирование, отмена действия
- Хранение данных в браузере: localStorage

JavaScript

JS – мультипарадигменный язык программирования. Поддерживает объектно-ориентированный, императивный и функциональный стили. Является реализацией языка ECMAScript (стандарт ECMA-262[8]).

- динамическая типизация
- слабая типизация
- автоматическое управление памятью
- прототипное программирование
- функции как объекты первого класса

Типы данных

- Число number
- Строка string
- Булевый (логический) тип boolean
- Специальное значение null
- Специальное значение undefined
- Объекты object

```
f = function() {  
    return 1  
}
```

JSON

JSON – **JavaScript Object Notation**, текстовый формат обмена данными. Его часто используют в HTTP API: сервер отдаёт JSON, браузер разбирает его в объект JavaScript.

JSON похож на объект JavaScript, но строже:

- ключи объекта всегда пишутся в двойных кавычках
- строки только в двойных кавычках
- нельзя писать комментарии
- нельзя использовать функции, `undefined`, `NaN`, `Infinity`
- в конце списка или объекта не должно быть лишней запятой

```
var a = []  
var b = {  
  "result": true,  
  "message": "hello",
```

JSON (ii)

```
  "data": [1, 2, 3, 4, 5]  
}
```

Переменные

use strict – строгий режим

```
"use strict";  
num = 5; // error: num is not defined
```

use strict должен располагаться в файле до кода.

```
var something;  
"use strict"; // слишком поздно  
num = 5; // ошибки не будет, так как строгий режим не  
активирован
```

Переменные

- Имя может состоять из: букв, цифр, символов \$ и _
- Первый символ не должен быть цифрой

```
var something;  
var COLOR_RED = "#F00";  
var myName;  
var $ = 1; // объявили переменную с именем '$'  
var _ = 2; // переменная с именем '_'  
alert($ + _); // 3
```


Определение функции

Функцию можно объявить через `function declaration` или сохранить в переменную как значение.

```
function sum(a, b) {  
    return a + b;  
}  
  
const multiply = function(a, b) {  
    return a * b;  
};  
  
console.log(sum(2, 3));           // 5  
console.log(multiply(2, 3));     // 6
```

let и const

В современном JavaScript var почти не используют.

- const – переменная не переназначается
- let – значение будет меняться
- область видимости у let и const ограничена блоком { ... }

```
const apiUrl = "/api/articles/";

let page = 1;
page = page + 1;

if (page > 1) {
    const message = "Следующая страница";
    console.log(message);
}
```

Стрелочные функции

Стрелочные функции часто используют в обработчиках событий, промисах и методах массивов.

```
const sum = (a, b) => a + b;

const showMessage = message => {
  console.log(message);
};

button.addEventListener("click", () => {
  console.log("Кнопка нажата");
});
```

Стрелочные и обычные функции

- обычная функция получает свой `this` при вызове
- стрелочная функция не имеет своего `this`, а берёт его из внешней области видимости
- у стрелочной функции нет собственного `arguments`
- стрелочную функцию нельзя использовать как конструктор через `new`

```
const user = {  
  name: "Анна",  
  
  sayName: function() {  
    console.log(this.name);  
  },  
  
  sayNameLater: function() {  
    setTimeout(() => {  
      console.log(this.name);  
    }, 1000);  
  }  
};
```

Стрелочные и обычные функции (ii)

```
    }, 1000);  
  },  
};
```

```
user.sayName();           // Анна  
user.sayNameLater();      // Анна
```

Основные операторы

Унарные операторы

```
var x = 1;  
x = -x;  
alert(x); //
```

Бинарные операторы

```
var x = 1;  
var y = 2;  
x = x + y;  
alert(x); //
```

Сложение строк

```
var a = "my" + "string";  
alert(a); //
```

```
alert("1" + 2); // "12"  
alert(2 + "1"); // "21"
```

```
alert(2 - "1"); // 1  
alert(6 / "2"); // 3
```

Сложение строк

```
var apples = "2";  
var oranges = "3";  
  
alert(apples + oranges); //
```

Используем унарный плюс, чтобы преобразовать к числу:

```
var apples = "2";  
var oranges = "3";  
  
alert(+apples + +oranges);
```


Логические операции

- Больше/меньше: $a > b$, $a < b$
- Больше/меньше или равно: $a \geq b$, $a \leq b$
- Равно $a == b$. Для сравнения используется два символа равенства $=$.
Один символ $a = b$ означал бы присваивание
- Не равно. В JavaScript пишется как знак равенства с восклицательным знаком перед ним: $!=$

```
alert(2 > 1); // true,  
alert(2 == 1); // false,  
alert(2 != 1); // true
```

Логические операции

```
alert("Вася" > "Ваня"); // true, т.к. "с" > "н"  
alert("Привет" > "Прив"); // true, так как "е" больше чем  
"ничего".
```

```
alert("2" > "14"); // true, неверно, ведь 2 не больше 14
```

Логические операции

При сравнении значений разных типов, используется числовое преобразование. Оно применяется к обоим значениям.

```
alert("2" > 1); // true, сравнивается как 2 > 1
alert("01" == 1); // true, сравнивается как 1 == 1
alert(false == 0); // true, false становится числом 0
alert(true == 1); // true, так как true становится числом 1.
```

Логические операции ==

```
alert(0 == false); // true
```

```
alert("" == false); // true
```

```
alert(0 === false); // false, т.к. типы различны
```

== – означает типа равно

=== – а это точно равно

Логические операции null и undefined

Значения null и undefined равны == друг другу и не равны чему бы то ни было ещё.

При преобразовании в число null становится 0, а undefined становится NaN.

```
alert(null > 0); // false  
alert(null == 0); // false
```

Итак, мы получили, что null не больше и не равен нулю. А теперь...

```
alert(null >= 0); // true
```

```
alert(undefined > 0); // false (1)  
alert(undefined < 0); // false (2)  
alert(undefined == 0); // false (3)
```

Логические операции: вопросительный знак

условие ? значение1 : значение2

```
access = (age > 14) ? true : false;  
access = age > 14 ? true : false;
```

```
if ("0") {  
    alert('Привет');  
}
```

fetch

fetch выполняет HTTP-запрос из браузера. Это базовый способ обращаться к Django API.

```
fetch("/api/articles/")  
  .then(response => response.json())  
  .then(data => {  
    console.log(data);  
  })  
  .catch(error => {  
    console.error("Ошибка запроса", error);  
  });
```

async/await

async/await делает асинхронный код похожим на обычный последовательный код.

```
async function loadArticles() {  
  try {  
    const response = await fetch("/api/articles/");  
    const articles = await response.json();  
    console.log(articles);  
  } catch (error) {  
    console.error("Ошибка запроса", error);  
  }  
}  
  
loadArticles();
```


Замыкания

Замыкание – это функция вместе с переменными из внешней области видимости, к которым она продолжает иметь доступ после завершения внешней функции.

```
function createCounter() {  
  let count = 0;  
  
  return function() {  
    count = count + 1;  
    return count;  
  };  
}  
  
const counter = createCounter();  
  
console.log(counter()); // 1
```

Замыкания (ii)

```
console.log(counter()); // 2  
console.log(counter()); // 3
```

Шаблонные строки

Шаблонные строки пишутся в обратных кавычках и позволяют вставлять выражения через `${...}`.

```
const user = "Анна";  
const count = 5;  
  
const message = `Привет, ${user}. Новых сообщений: ${count}`;
```

Методы массивов

map, filter, find часто встречаются во frontend-коде и во Vue.

```
const users = [  
  { id: 1, name: "Анна", active: true },  
  { id: 2, name: "Иван", active: false },  
  { id: 3, name: "Ольга", active: true },  
];  
  
const names = users.map(user => user.name);  
const activeUsers = users.filter(user => user.active);  
const ivan = users.find(user => user.name === "Иван");
```

Деструктуризация

Деструктуризация достаёт значения из объекта или массива в отдельные переменные.

```
const article = {  
  id: 10,  
  title: "Django и Vue",  
  author: "admin",  
};  
  
const { id, title } = article;  
  
const colors = ["red", "green", "blue"];  
const [firstColor, secondColor] = colors;
```

Spread и rest

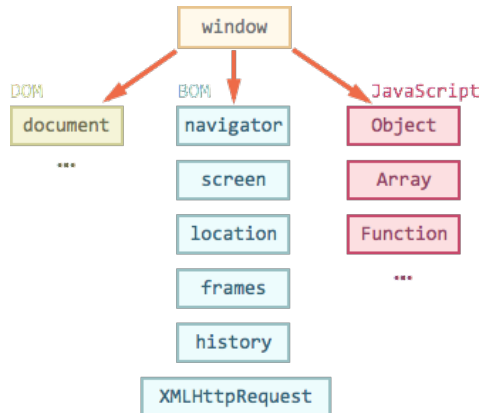
... используется для копирования, объединения и сбора оставшихся аргументов.

```
const user = { id: 1, name: "Анна" };
const updatedUser = { ...user, active: true };

const first = [1, 2];
const second = [3, 4];
const numbers = [...first, ...second];

function log(firstMessage, ...otherMessages) {
  console.log(firstMessage);
  console.log(otherMessages);
}
```

DOM



document.body

```
<!DOCTYPE HTML>
<html>

<head>
  <script>
    alert("Из HEAD: " + document.body); // null, body ещё нет
  </script>
</head>

<body>

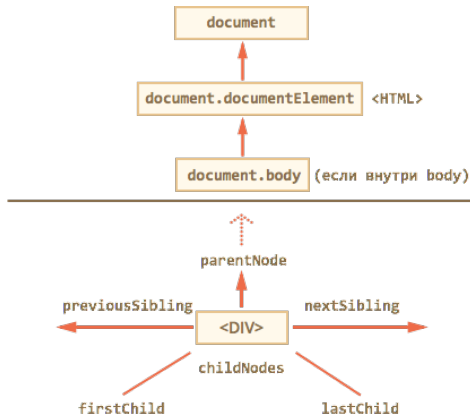
  <script>
    alert("Из BODY: " + document.body); // body есть
  </script>

</body>
```


document.body (ii)

```
</html>
```

Навигация по DOM-элементам



innerHTML

```
<body>
  <p>Параграф</p>
  <div>Div</div>

  <script>
    alert(document.body.innerHTML); // читаем текущее
содержимое
    document.body.innerHTML = "Новый BODY!"; // заменяем
содержимое
  </script>

</body>
```

outerHTML

```
<div>Привет <b>Мир</b></div>
```

```
<script>
```

```
  var div = document.body.children[0];
```

```
  alert(div.outerHTML); // <div>Привет <b>Мир</b></div>
```

```
</script>
```

Текст: textContent

```
<div>
  <h1>Срочно в номер!</h1>
  <p>Марсиане атакуют людей!</p>
</div>

<script>
  var news = document.body.children[0];

  // \n Срочно в номер!\n Марсиане атакуют людей!\n
  alert(news.textContent);
</script>
```

Свойство hidden

```
<div>Текст</div>
<div hidden>С атрибутом hidden</div>
<div>Со свойством hidden</div>

<script>
  var lastDiv = document.body.children[2];
  lastDiv.hidden = true;
</script>
```

Исследование элементов

```
<input type="text" id="elem" value="значение">
```

```
<script>
```

```
  var input = document.body.children[0];
```

```
  alert(input.type); // "text"
```

```
  alert(input.id); // "elem"
```

```
  alert(input.value); // значение
```

```
</script>
```


Создание элемента

```
var div = document.createElement("div");  
div.className = "alert alert-success";  
div.innerHTML = "<strong> Ура!</strong> Вы прочитали это важное  
сообщение.";
```

Добавление элемента

```
<ol id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>

<script>
  var newLi = document.createElement("li");
  newLi.innerHTML = "Привет, мир!";

  list.appendChild(newLi);
</script>
```

Метод `document.write`

Нет никаких ограничений на содержимое `document.write`.

```
<body>
  1
  <script>
    document.write(2);
  </script>
  3
</body>
```

Только до конца загрузки.

Получение элемента

```
<div id="content">Выделим этот элемент</div>
```

```
<script>
```

```
var elem = document.getElementById('content');
```

```
elem.style.background = 'red';
```

```
</script>
```

- `getElementsById`
- `getElementsByTagName`
- `getElementsByClassName`
- `querySelectorAll`
- `querySelector`

querySelector

querySelector возвращает первый элемент по CSS-селектору,
querySelectorAll – все подходящие элементы.

```
const form = document.querySelector("#article-form");
const titleInput =
document.querySelector("input[name='title']");
const buttons = document.querySelectorAll("button[data-
action]");

buttons.forEach(button => {
  console.log(button.dataset.action);
});
```

Стили элемента: свойство style

```
background-color => elem.style.backgroundColor  
z-index         => elem.style.zIndex  
border-left-width => elem.style.borderLeftWidth
```

Браузерные события

События мыши:

- `click` – происходит, когда кликнули на элемент левой кнопкой мыши
- `contextmenu` – происходит, когда кликнули на элемент правой кнопкой мыши
- `mouseover` – возникает, когда на элемент наводится мышь
- `mousedown` и `mouseup` – когда кнопку мыши нажали или отжали
- `mousemove` – при движении мыши

События на элементах управления:

- `submit` – посетитель отправил форму `<form>`
- `focus` – посетитель фокусируется на элементе, например нажимает на `<input>`

Клавиатурные события:

- `keydown` – когда посетитель нажимает клавишу

Браузерные события (ii)

- `keyup` – когда посетитель отпускает клавишу

События документа:

- `DOMContentLoaded` – когда HTML загружен и обработан, DOM документа полностью построен и доступен

DOMContentLoaded

Код, который работает с DOM, безопасно запускать после события DOMContentLoaded.

```
document.addEventListener("DOMContentLoaded", () => {  
    const button = document.querySelector("#save-button");  
  
    button.addEventListener("click", () => {  
        console.log("Сохранить");  
    });  
});
```

Назначение обработчиков событий

```
<input value="Нажми меня" onclick="alert('Клик!')"  
type="button">
```

```
<!DOCTYPE HTML>  
<html>  
<head>  
  <meta charset="utf-8">  
  
  <script>  
    function countRabbits() {  
      for (var i = 1; i <= 3; i++) {  
        alert("Кролик номер " + i);  
      }  
    }  
  </script>  
</head>
```

Назначение обработчиков событий (ii)

```
<body>  
  <input type="button" onclick="countRabbits()" value="Считать  
кроликов!"/>  
</body>  
</html>
```

Назначение обработчиков событий

```
<input value="Нажми меня" onclick="alert('Клик!')"  
type="button">
```

```
<input id="elem" type="button" value="Нажми меня" />  
<script>  
  elem.onclick = function() {  
    alert("Спасибо");  
  };  
</script>
```

Доступ к элементу через this.

```
<button onclick="alert(this.innerHTML)">Нажми меня</button>
```

addEventListener и removeEventListener

```
element.addEventListener(event, handler[, phase]);
```

- event – имя события, например click
- handler – ссылка на функцию, которую надо поставить обработчиком
- phase – необязательный аргумент, «фаза», на которой обработчик должен сработать

Всплытие

При наступлении события обработчики сначала срабатывают на самом вложенном элементе, затем на его родителе, затем выше и так далее, вверх по цепочке вложенности.

```
<style>
  body * {
    margin: 10px;
    border: 1px solid blue;
  }
</style>

<form onclick="alert('form')">FORM
  <div onclick="alert('div')">DIV
    <p onclick="alert('p')">P</p>
  </div>
</form>
```

Делегирование событий

Делегирование удобно, когда элементы создаются динамически: обработчик ставится на родителя.

```
<ul id="todo-list">
  <li><button data-id="1">Удалить</button> Купить хлеб</li>
  <li><button data-id="2">Удалить</button> Сделать проект</li>
</ul>

<script>
  const list = document.querySelector("#todo-list");

  list.addEventListener("click", event => {
    if (!event.target.matches("button[data-id]")) {
      return;
    }

    console.log("Удалить", event.target.dataset.id);
```

Делегирование событий (ii)

```
});  
</script>
```


Отмена действия браузера

Любой промежуточный обработчик может решить, что событие полностью обработано, и остановить всплытие.

```
<a href="/" onclick="return false">Нажми здесь</a>
```

или

```
<a href="/" onclick="event.preventDefault()">здесь</a>
```

localStorage

localStorage хранит строки в браузере между перезагрузками страницы.

```
localStorage.setItem("theme", "dark");

const theme = localStorage.getItem("theme");

if (theme === "dark") {
    document.body.classList.add("dark-theme");
}

localStorage.removeItem("theme");
```